JAVA

By Arijit Chowdhury

BACKGROUND

Java is a high-level, general-purpose, memory-safe, object-oriented programming language. It is intended to let programmers write once, run anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need to recompile.

OOP

All computer programs consist of two elements: code and data.

That is, some programs are written around “what is happening” and others are written around “who is being affected.”

The first way is called the process-oriented model. The process-oriented model can be thought of as code acting on data. with this approach appear as programs grow larger and more complex.

The second approach, called object-oriented programming, was conceived. Object-oriented programming organizes a program around its data (that is, objects) and a set of well-defined interfaces to that data. An object-oriented program can be characterized as data controlling access to code. By switching the controlling entity to data, you can achieve several organizational benefits

ABSTRACTION

An essential element of object-oriented programming is abstraction. Humans manage complexity through abstraction. For example, people do not think of a car as a set of tens of thousands of individual parts. They think of it as a well-defined object with its own unique behavior. This abstraction allows people to use a car to drive to the grocery store without being overwhelmed by the complexity of the parts that form the car. They can ignore the details of how the engine, transmission, and braking systems work. Instead, they are free to utilize the object as a whole. A powerful way to manage abstraction is through the use of hierarchical classifications. This allows you to layer the semantics of complex systems, breaking them into more manageable pieces. From the outside, the car is a single object.

THE THREE OOP PRINCIPLES

All object-oriented programming languages provide mechanisms that help you implement the object-oriented model. They are encapsulation, inheritance, and polymorphism

ENCAPSULATION

Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse. One way to think about encapsulation is as a protective wrapper that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper. In Java, the basis of encapsulation is the class. A class defines the structure and behavior (data and code) that will be shared by a set of objects. objects are sometimes referred to as instances of a class. When you create a class, you will specify the code and data that constitute that class. Collectively, these elements are called members of the class. Specifically, the data defined by the class are referred to as member variables. The code that operates on that data is referred to as member methods or just methods.(If you are familiar with C/C++, it may help to know that what a Java programmer calls a method, a C/C++ programmer calls a function.)Since the purpose of a class is to encapsulate complexity, there are mechanisms for hiding the complexity of the implementation inside the class. Each method or variable in a class may be marked private or public. The public interface of a class represents everything that external users of the class need to know, or may know. The private methods and data can only be accessed by code that is a member of the class.

INHERITANCE

Inheritance is the process by which one object acquires the properties of another object. This is important because it supports the concept of hierarchical classification. a Golden Retriever is part of the classification dog, which in turn is part of the mammal class, which is under the larger class animal. Without the use of hierarchies, each object would need to define all of its characteristics explicitly.

POLYMORPHISM

Polymorphism (from Greek, meaning “many forms”) is a feature that allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation. Consider a stack (which is a last-in, first-out list). You might have a program that requires three types of stacks. One stack is used for integer values, one for floatingpoint values, and one for characters. The algorithm that implements each stack is the same, even though the data being stored differs. In a non–object-oriented language, you would be required to create three different sets of stack routines, with each set using different names. However, because of polymorphism, in Java you can specify a general set of stack routines that all share the same names. Extending the dog analogy, a dog’s sense of smell is polymorphic. If the dog smells a cat, it will bark and run after it. If the dog smells its food, it will salivate and run to its bowl. The same sense of smell is at work in both situations.

POLYMORPHISM, ENCAPSULATION, AND INHERITANCE WORK TOGETHER

When properly applied, polymorphism, encapsulation, and inheritance combine to produce a programming environment that supports the development of far more robust and scaleable programs than does the process-oriented model. A well-designed hierarchy of classes is the basis for reusing the code in which you have invested time and effort developing and testing. Encapsulation allows you to migrate your implementations over time without breaking the code that depends on the public interface of your classes. Polymorphism allows you to create clean, sensible, readable, and resilient code.

A FIRST SIMPLE PROGRAM

```
class FileName{
    public static void main(String[] args) {
        System.out.println("hello");
    }
}
```

In Java, a source file is officially called a compilation unit. It is a text file that contains (among other things) one or more class definitions. (For now, we will be using source files that contain only one class.) The Java compiler requires that a source file use the .java filename extension.

To compile the Example program, execute the compiler, javac, specifying the name of the source file on the command line, as shown here:
E:\Java>javac src/Main.java
The javac compiler creates a file called Main.class that contains the bytecode version of the program. The Java bytecode is the intermediate representation of your program that contains instructions the Java Virtual Machine will execute. Thus, the output of javac is not code that can be directly executed.

To actually run the program, you must use the Java application launcher called java. To do so, pass the class name Example as a command-line argument, as shown here: C:\>java -cp src Main
When the program is run, the following output is displayed: **hello**.
When Java source code is compiled, each individual class is put into its own output file named after the class and using the .class extension. This is why it is a good idea to give your Java source files the same name as the class they contain—the name of the source file will match the name of the .class file. When you execute java as just shown, you are actually specifying the name of the class that you want to execute. It will automatically search for a file by that name that has the .class extension. If it finds the file, it will execute the code contained in the specified class.

COMMENTS

Java programming language supports three types of comments:

- Single-line Comments:
  - Syntax: // comment_text
  - Purpose: Used for short, one-line explanations or notes within the code. The comment extends from the // to the end of the line.
- Multi-line Comments (Block Comments):
  - Syntax: /* comment_text */
  - Purpose: Used for longer explanations that span multiple lines, or to temporarily comment out blocks of code during development or debugging. The comment starts with /* and ends with */.
- Documentation Comments (Javadoc Comments):
  - Syntax: /** comment_text */
  - Purpose: Used to generate API documentation automatically using the Javadoc tool.

PROGRAM

class Main {

This line uses the keyword class to declare that a new class is being defined. Main is an identifier that is the name of the class. The entire class definition, including all of its members, will be between the opening curly brace ({) and the closing curly brace (}).

public static void main(String args[ ]) {

This line begins the main( ) method. The public keyword is an access modifier, which allows the programmer to control the visibility of class members. When a class member is preceded by public, then that member may be accessed by code outside the class in which it is declared. Any information that you need to pass to a method is received by variables specified within the set of parentheses that follow the name of the method. These variables are called parameters. String args[ ] declares a parameter named args, which is an array of instances of the class String.

System.out.println("This is a simple Java program.");

Output is actually accomplished by the built-in println( ) method. In this case, println( ) displays the string which is passed to it. As you will see, println( ) can be used to display other types of information, too. The line begins with System.out. System is a predefined class that provides access to the system, and out is the output stream that is connected to the console.

SECOND PROGRAM

do Add, sub, mul, div etc.

CONTROL STATEMENTS

The if Statement:
if(condition) statement;
Here, condition is a Boolean expression. If condition is true, then the statement is executed.
If condition is false, then the statement is bypassed.

ex:voteable age or not

FOR LOOP

```
for(initialization; condition; iteration)
statement;
int x;
for(x = 0; x<10; x = x+1)
System.out.println("This is x: " + x);
```

SEPARATORS

Symbol	Name	Purpose
( )	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[ ]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.

SEPARATORS

Symbol	Name	Purpose
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.

JAVA KEYWORDS

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

THANK YOU